BOOTCAMP II

PLANEJAMENTO LÓGICO TASKTRACKER

Aluno: João Pedro Neves Queiroz

Curso: Análise e Desenvolvimento de Sistemas

Disciplina: Bootcamp II

Documento de planejamento para a implementação futura do sistema TaskTracker.

1. Identificação e Entendimento do Problema

1.1 Identificação

Nome	João Pedro Neves Queiroz
Curso	Análise e Desenvolvimento de Sistemas
Disciplina	Bootcamp II
E-mail institucional	joao.pq@sempreceub.com

1.2 O problema a ser resolvido

Pessoas e equipes frequentemente possuem diversas tarefas que precisam ser realizadas durante o dia, mas podem ter dificuldade para organizar, priorizar e acompanhar essas atividades.

Sem um sistema de gerenciamento, as tarefas podem ser esquecidas, realizadas fora de ordem ou não receber a devida prioridade. Também pode ser difícil visualizar rapidamente quais atividades estão cadastradas e quais precisam de maior atenção.

1.3 A solução proposta — TaskTracker

O TaskTracker será um sistema simples de gerenciamento de tarefas, desenvolvido inicialmente em Python.

O sistema permitirá que o usuário cadastre tarefas informando seu título e prioridade. As tarefas serão armazenadas em uma lista e poderão posteriormente ser visualizadas pelo usuário.

A proposta é oferecer uma solução simples para cadastrar, organizar e visualizar tarefas, servindo também como base para a evolução do projeto nas próximas etapas do Bootcamp.

2. Decomposição do Problema e Dados do Sistema

2.1 Entradas de dados

Dado	Descrição	Exemplo
Título	Nome ou descrição resumida da tarefa.	Estudar Python
Prioridade	Grau de importância da tarefa.	Alta
Status	Situação da tarefa, quando aplicável.	Pendente

2.2 Saídas de dados

- Mensagem de confirmação após o cadastro de uma tarefa.
- Lista das tarefas cadastradas.

- Título de cada tarefa.
- Prioridade de cada tarefa.
- Status da tarefa, caso utilizado.
- Mensagens de erro ou orientação quando uma informação obrigatória não for preenchida.

2.3 Regras do sistema

Regra 1 — Título obrigatório: O sistema não deve permitir o cadastro de uma tarefa caso o título esteja vazio.

Regra 2 — Prioridade obrigatória: Toda tarefa cadastrada deve possuir uma prioridade.

Regra 3 — Prioridades válidas: A prioridade deve ser classificada como Alta, Média ou Baixa.

Regra 4 — Armazenamento: Uma tarefa somente deve ser adicionada à lista após a validação dos seus dados obrigatórios.

Regra 5 — Visualização: Quando não houver tarefas cadastradas, o sistema deve informar ao usuário que a lista está vazia.

3. Receita Lógica do Sistema (Algoritmo em Português)

3.1 Algoritmo de cadastro de tarefa

1. Iniciar o sistema.
2. Solicitar ao usuário o título da tarefa.
3. Verificar se o título foi preenchido.
4. Caso o título esteja vazio, informar o erro e solicitar novamente.
5. Solicitar ao usuário a prioridade da tarefa.
6. Verificar se a prioridade informada é válida.
7. Caso a prioridade seja inválida, informar o erro e solicitar novamente.
8. Criar a tarefa utilizando o título e a prioridade informados.
9. Definir o status inicial da tarefa como “Pendente”.
10. Adicionar a tarefa à lista de tarefas.
11. Exibir uma mensagem informando que a tarefa foi cadastrada com sucesso.

Fim.

3.2 Algoritmo de visualização das tarefas

12. Acessar a lista de tarefas cadastradas.
13. Verificar se existe alguma tarefa na lista.
14. Caso não exista nenhuma tarefa, informar ao usuário que não há tarefas cadastradas.
15. Caso existam tarefas, percorrer a lista.
16. Exibir o título de cada tarefa.
17. Exibir a prioridade correspondente.
18. Exibir o status correspondente.
19. Continuar até que todas as tarefas sejam apresentadas.

Fim.

3.3 Fluxo geral do sistema

```
INÍCIO
↓
Iniciar sistema
↓
Solicitar título da tarefa
↓
Título está preenchido?
├── NÃO → Informar erro → Solicitar novamente
└── SIM
    ↓
Solicitar prioridade
    ↓
Prioridade é válida?
├── NÃO → Informar erro → Solicitar novamente
└── SIM
    ↓
Criar tarefa
    ↓
Definir status como "Pendente"
    ↓
Adicionar tarefa à lista
    ↓
Informar sucesso
    ↓
Exibir tarefas quando solicitado
    ↓
FIM
```

4. Mapeamento Arquitetural para Git e GitHub

4.1 Árvore do repositório

```
tasktracker/
│
├── README.md
├── .gitignore
│
├── docs/
│   └── planejamento_logico.pdf
│
└── src/
    └── main.py
```

4.2 Descrição dos componentes

README.md — Arquivo responsável por apresentar o projeto, seu objetivo, instruções básicas e informações relevantes para utilização.

.gitignore — Arquivo responsável por indicar arquivos e diretórios que não devem ser enviados ao GitHub.

docs/ — Diretório destinado aos documentos de planejamento e documentação do projeto.

planejamento_logico.pdf — Documento produzido nesta primeira etapa, contendo a análise do problema, dados, regras, algoritmo e planejamento arquitetural.

src/ — Diretório destinado ao código-fonte do projeto.

main.py — Arquivo que posteriormente conterá a implementação principal do TaskTracker em Python.

4.3 Plano de aprendizado de Git e GitHub

Objetivo: Aprender e aplicar os principais recursos do Git e GitHub necessários para desenvolver e versionar o TaskTracker.

Controle de versões

- Compreender o funcionamento do Git.
- Criar um repositório.
- Realizar commits.
- Consultar o histórico de alterações.

Repositórios no GitHub

- Criar e configurar um repositório remoto.
- Conectar o projeto local ao GitHub.
- Realizar push e pull.
- Organizar os arquivos do projeto.

Branches

- Criar branches para novas funcionalidades.
- Trabalhar de forma isolada sem alterar diretamente a versão principal.
- Realizar merge das alterações.

Resolução de conflitos

- Identificar conflitos entre versões.
- Resolver conflitos manualmente.
- Realizar novo commit após a resolução.

Organização do projeto

- Manter o código-fonte separado da documentação.
- Utilizar um .gitignore.
- Manter o README.md atualizado.